KUMI

A private, portable workspace

Product, UI/UX, architecture, migration, and acceptance specification for a Notion-compatible clone

Prepared from a read-only exploration of the logged-in AMP Notion workspace, supported by current official Notion export, import, and API documentation.

Document	Value
Status	Implementation-ready baseline
Observation date	19 August 2026
Target	Responsive web app; desktop-first
Primary goal	Migrate away from Notion without losing structure or day-to-day workflows
Working product name	Kumi

Independent implementation note

Reproduce interaction patterns and interoperability, not Notion trademarks, logos, proprietary copy, or visual assets. Ship an original Kumi identity and verify applicable terms, licenses, and export rights before commercial release.

1. Executive specification

Build a self-hostable workspace that feels immediately familiar to a Notion user, but treats portability, transparent storage, and recoverable exports as core product features.

Outcome

A user can import a Notion workspace export, continue working in a familiar page-and-database interface, collaborate safely, find content quickly, and export everything again in documented, non-proprietary formats. The first release must prioritize fidelity of user data and editor/database workflows over AI or marketplace breadth.

Success measures

- At least 99.9% of supported blocks and database property values import without silent loss; every unsupported item appears in a downloadable loss report.
- A 10,000-page test workspace imports resumably, with stable IDs and internal links preserved or explicitly remapped.
- Common interactions - open page, edit text, switch database view, filter, search, share, undo - are reachable with the same conceptual model and comparable keyboard efficiency.
- A complete workspace export can be opened without Kumi: Markdown/HTML for pages, CSV or JSON for data, an assets directory, and a machine-readable manifest.
- The desktop shell reaches usable content within 2 seconds at p75 on a warm session; local editor input feedback stays below 50 ms at p95.

Priority legend

Level	Meaning	Release rule
P0	Required for migration and daily use	Blocks launch
P1	Important parity or collaboration	Target first two post-launch increments
P2	Expansion and differentiation	Only after portability and reliability gates pass

2. Observed product model

The app is a workspace graph presented through three coordinated layers: a persistent navigation shell, URL-addressable pages made of blocks, and typed data sources rendered through reusable views.

Layer	Observed behavior	Kumi interpretation
Workspace shell	Approx. 270 px hierarchy rail, 44 px page header, flexible canvas, global search, inbox and bottom New action.	Persistent application state; keyboard-aware; collapsible; responsive drawer below desktop width.
Page	Icon, cover, title, breadcrumbs, blocks, comments, share, actions, favorite and full-page/peek modes.	Stable entity with parent edge, ACL, block tree, properties, history and canonical route.
Block	Rich-text and structural units with hover handles, slash insertion, nesting and reorder semantics.	Ordered tree nodes, individually addressable and versioned; optimistic edits.
Data source	Typed schema; rows are pages; inline or full-page rendering; multiple saved views.	Relational model exposed through user-defined schema and reusable query/view definitions.
View	Table, board, gallery, list, chart, dashboard, timeline, feed, map, calendar and form.	Non-destructive projection: layout + filter + sort + group + visible properties + lock state.
Migration	Broad import hub and workspace export control; file and third-party sources.	Durable job pipeline with parsing, resolution, verification, retry and loss reporting.

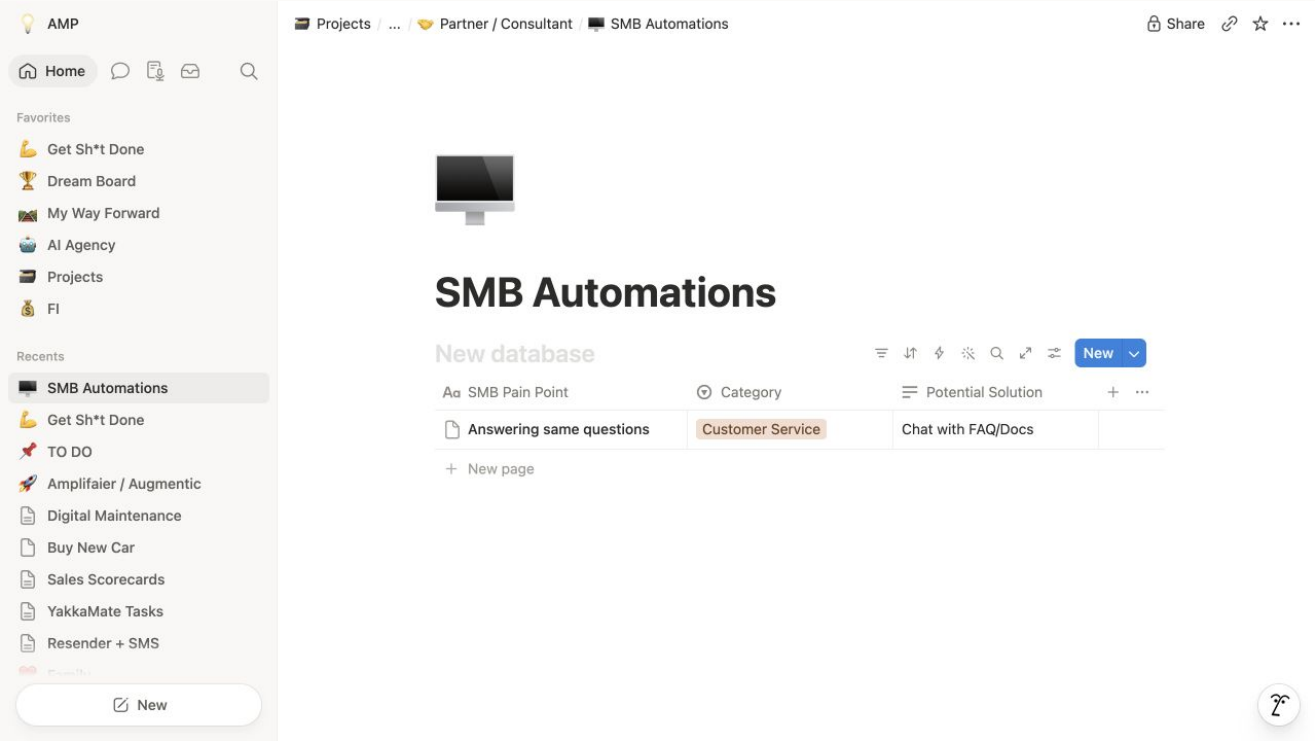


Figure 1. Observed desktop shell: persistent left rail, breadcrumb header, page title, inline table data source, compact query toolbar, and primary New action.

3. Information architecture and navigation

P0 Workspace and identity

- Support multiple workspaces per account, a visible active workspace, switcher, member list and workspace-level settings.
- Workspace roles: owner, admin, member and guest. Separate billing/admin privileges from content permissions.
- Every workspace has a stable ID, name, icon, locale, timezone, export policy and retention policy.

P0 Sidebar hierarchy

- Sections: Home, Inbox/activity, Search, Favorites, Recents, team/shared/private page trees, Trash, Settings and a persistent New button.
- Tree nodes support expand/collapse, hover actions, add child, reorder, favorite, context menu and unread/update badges.
- Persist expansion and scroll state per user and workspace; never let search navigation destroy the user's prior context.

P0 Routes and open modes

- Canonical page URLs use immutable IDs and slug-like labels for readability. Renames do not break links.
- Open database items as center peek, side peek or full page. Back/forward navigation must restore the exact previous view and scroll position.
- Breadcrumbs reflect hierarchy and allow keyboard and pointer navigation to ancestors.

P1 Home, recents and favorites

- Home aggregates recently visited pages, assigned work, saved views and workspace announcements.
- Recents are per-user, ordered by last access; favorites are explicitly ordered and sync across devices.

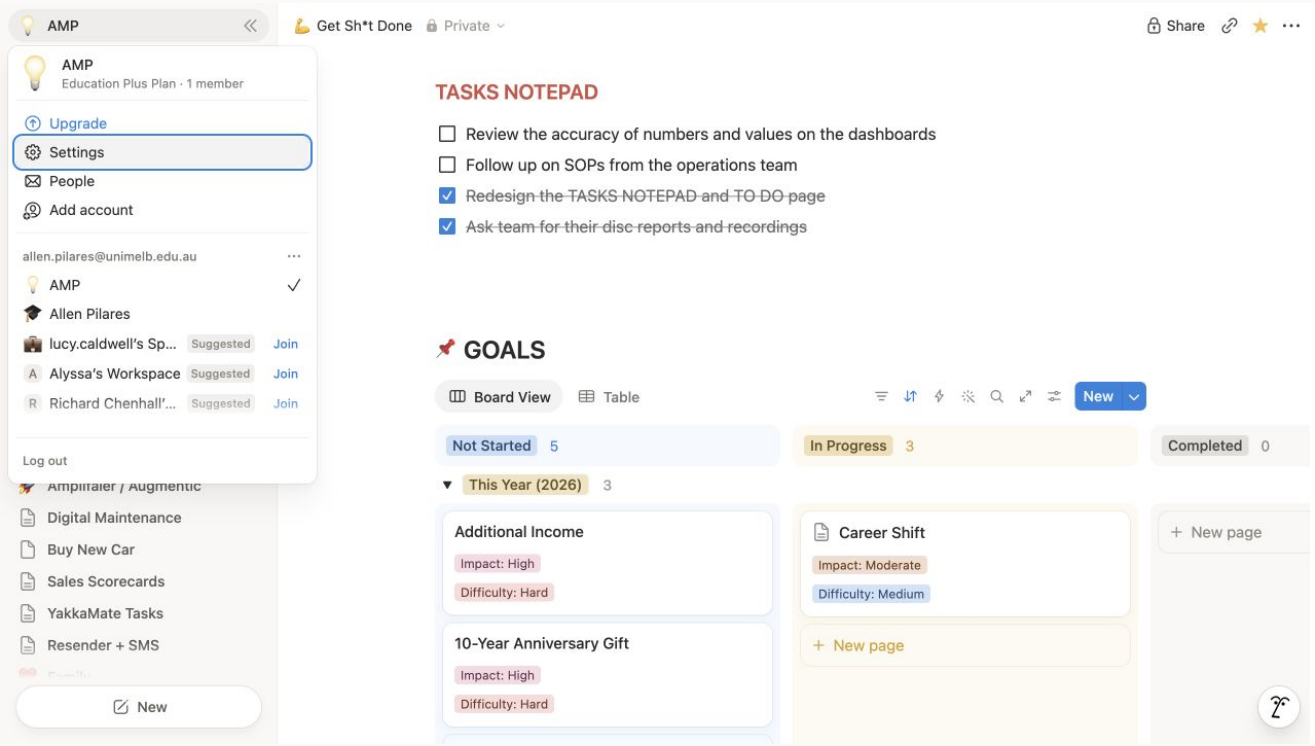


Figure 2. Workspace switcher reference: active account, workspace list, settings/people entry points, account addition and sign-out controls.

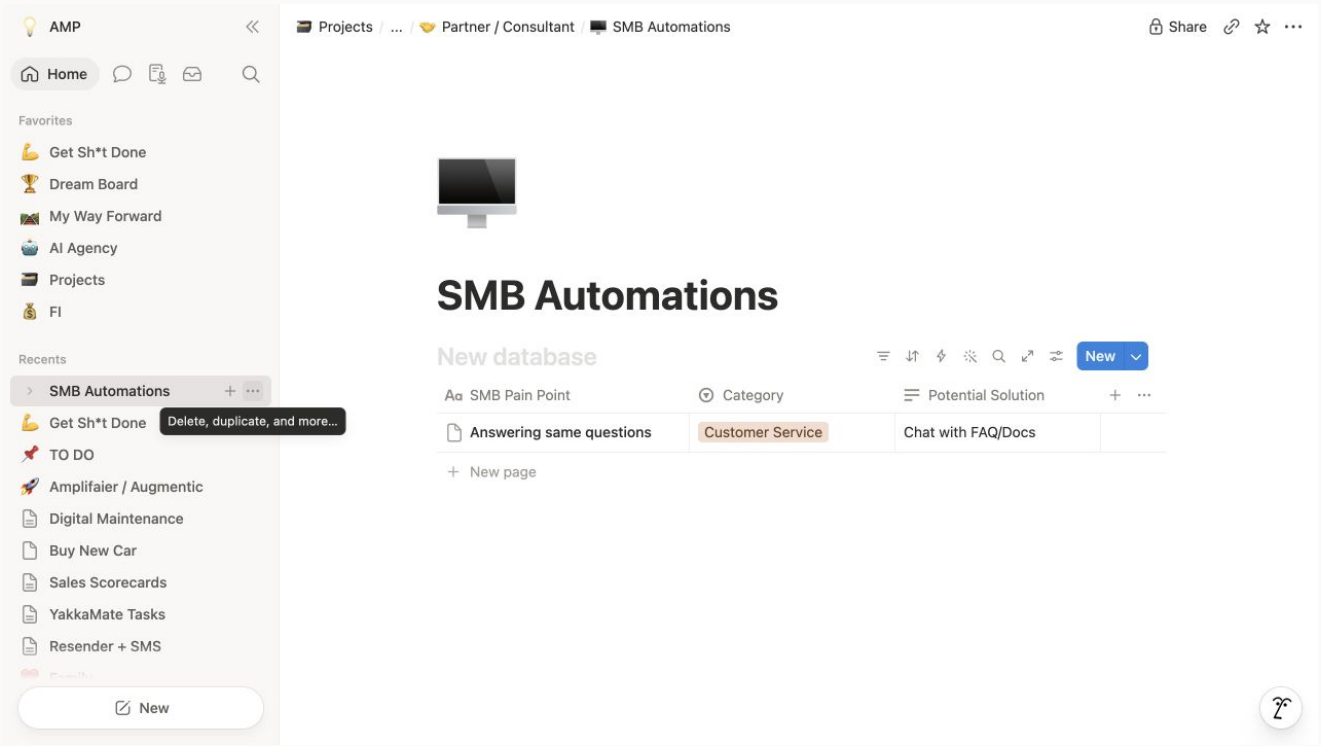


Figure 3. Verified tree affordances: a hovered page exposes open, add-child and more actions while retaining the hierarchy and current-page context.

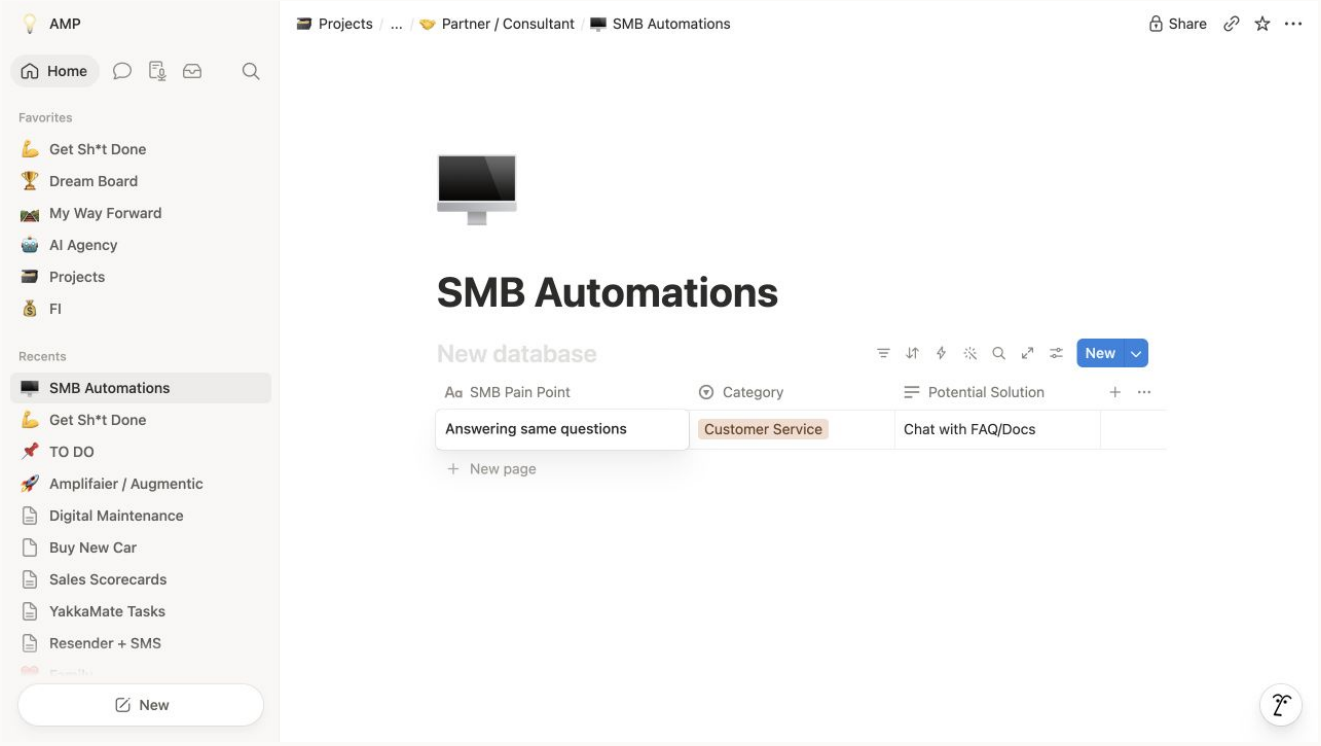


Figure 4. Verified database-row activation from the originating table. A row is a page entity and can open in a transient peek while preserving database context.

4. Page editor requirements

The editor is the primary system of record. Build it as a structured block tree with rich-text spans, not as stored HTML.

P0 Document foundation

- Page fields: ID, workspace, parent, title, icon, cover, created/edited metadata, creator/editor, state, lock state and ACL reference.
- Block fields: ID, page, parent block, ordered rank, type, typed payload, created/edited metadata, creator/editor, archived flag and version.
- Use fractional ordering or equivalent to make reorder operations stable under concurrent editing.

P0 P0 block types

- Paragraph; headings 1-3; bulleted/numbered lists; to-do; toggle; quote; callout; divider; code; equation; bookmark; table of contents.
- Image, file, video/audio, embed, link preview; child page; linked page; columns; simple table; synced block reference.
- Inline mentions for people, pages, dates and databases; links; bold/italic/underline/strike/code/color annotations.

P0 Editing interactions

- Slash command menu, Markdown shortcuts, drag handle, duplicate/delete/turn-into/color actions, multi-block selection, drag-and-drop and paste conversion.
- Command-key shortcuts, contextual formatting toolbar, undo/redo, find in page, block link copy, comments and suggestion mode.
- Autosave with local optimistic updates and explicit sync/error indicators; keyboard focus must never jump after reconciliation.

P1 History and recovery

- Version snapshots and an operation log; restore creates a new version instead of destroying later history.
- Page lock prevents normal edits but still permits authorized unlock; Trash is recoverable with retention settings.

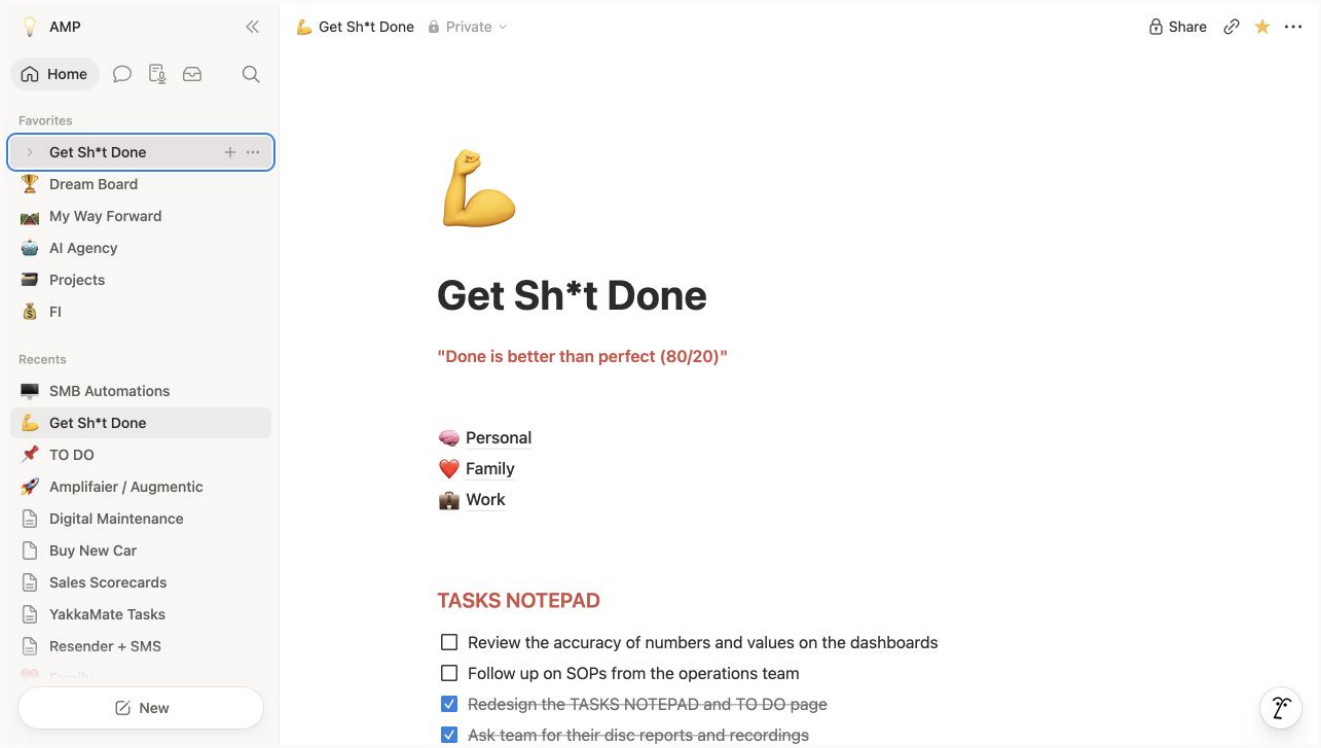


Figure 5. Rich page reference: large title rhythm, linked child pages, headings, to-do blocks and an inline database, with low-chrome editing affordances.

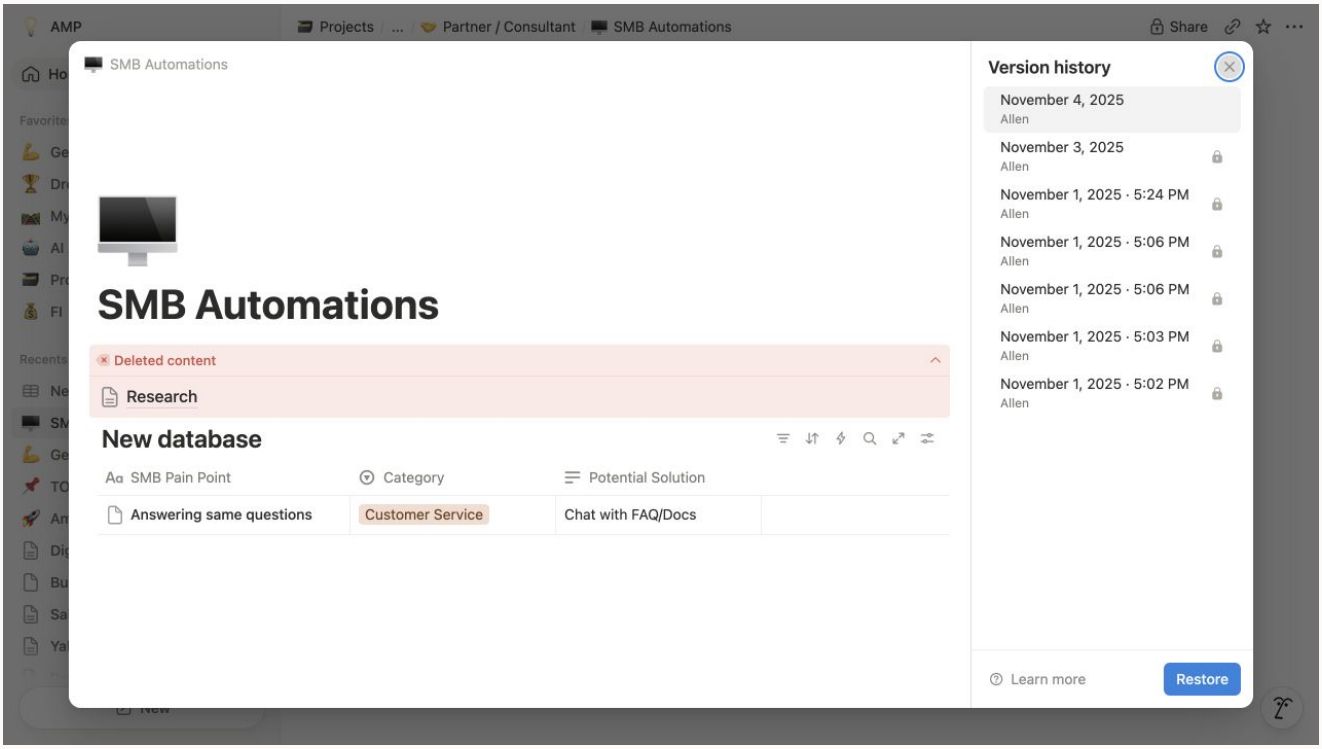


Figure 6. Verified page history: dated revisions and authors are inspectable before an explicit Restore operation.

5. Database and saved-view requirements

Rows are pages, properties are typed schema, and views are query/layout definitions. Never flatten this model to a spreadsheet-only abstraction.

Property types

Type	Storage	Required behavior
Title	Rich text spans	Exactly one per data source; opens the row page.
Text	Rich text spans	Formatting, mentions and links.
Number	Decimal + format	Currency, percentage, separators and precision.
Select/status	Stable option IDs	Color, name, ordering; status groups.
Multi-select	Option ID set	Order-insensitive values, stable options.
Date	Start/end + timezone	Date-only, datetime, ranges and reminders.
Person	Member ID set	Guests and deleted users remain referentially safe.
Files	Asset references	Multiple files, captions, durable object storage.
Checkbox/URL/email/phone	Typed scalar	Validation without destructive coercion.
Relation	Page ID set	Bidirectional optional mirror; deferred import resolution.
Rollup	Derived expression	Supported aggregation over relations.
Formula	Parsed expression	Deterministic evaluation and dependency graph.
Created/edited metadata	System value	Immutable creation data; audited edits.
Unique ID	Prefix + counter	Workspace-safe, stable and not reused.

View contract

- A view stores data source ID, layout type, filter tree, sort list, group rules, visible property order, property display options, conditional colors, row/page open mode and lock state.
- Filters support nested AND/OR groups and operators appropriate to each property type. Dates include relative periods. Person filters include current user.
- Sorts are stable and ordered; group and subgroup can collapse independently and maintain per-user display state.
- Editing a row mutates the shared data source; changing a view only mutates that view unless schema changes are explicitly chosen.

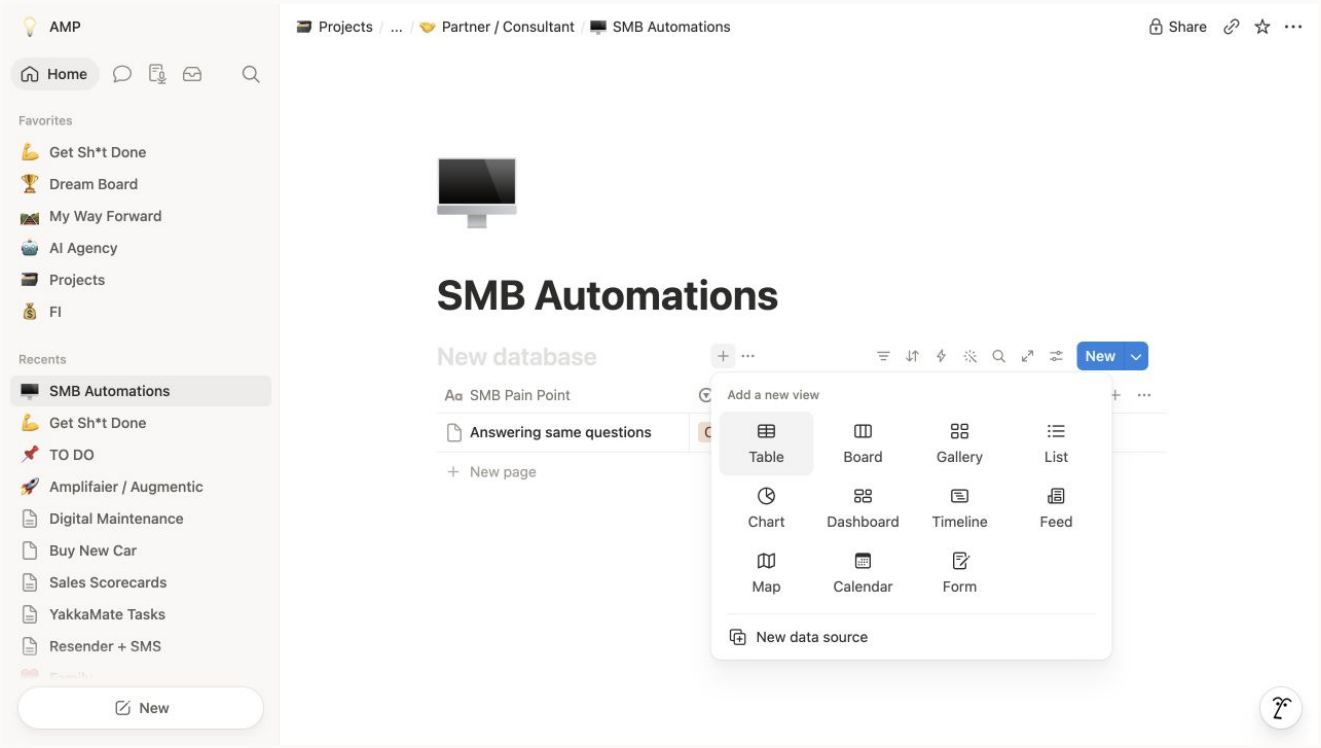


Figure 7. Observed add-view menu. P0 ships table, board, gallery, list, calendar and timeline; chart, dashboard, feed, map and form may follow without changing the data model.

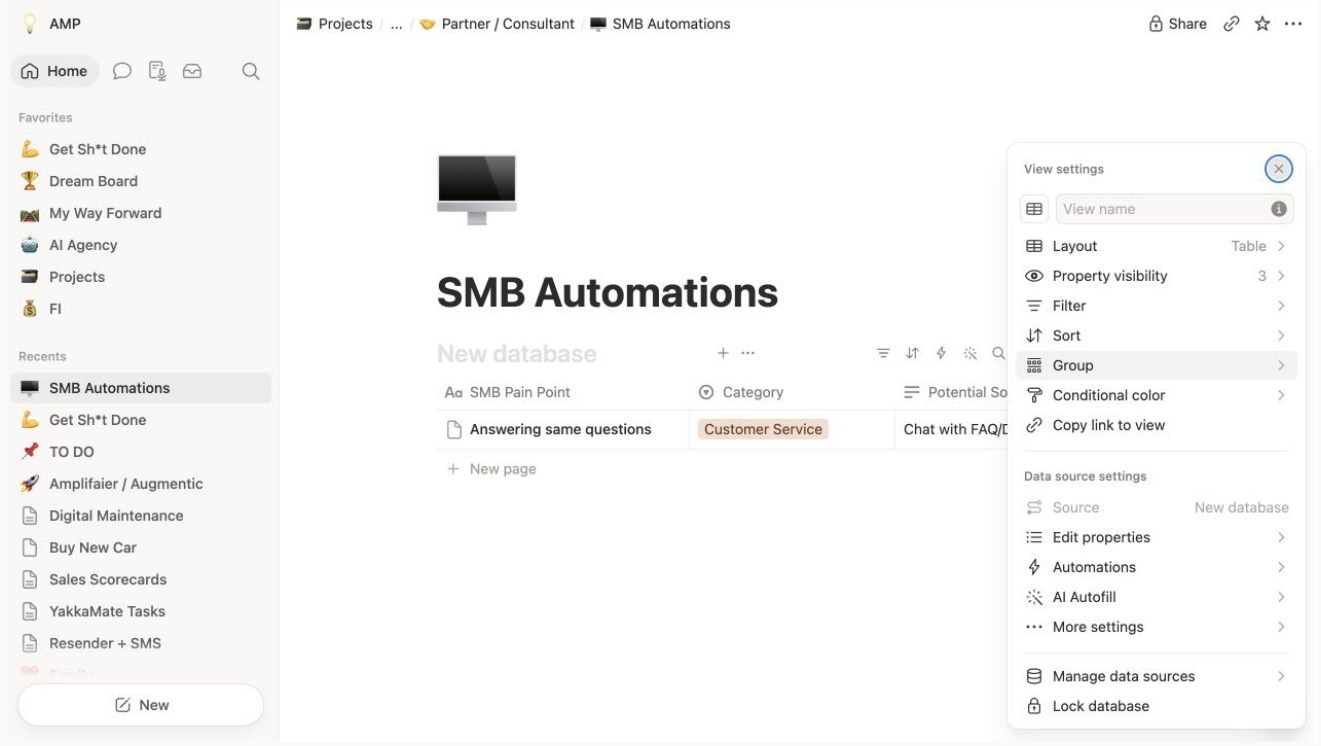


Figure 8. View settings are a layered inspector: layout, property visibility, filter, sort, grouping, color, source/schema controls, automation and locking.

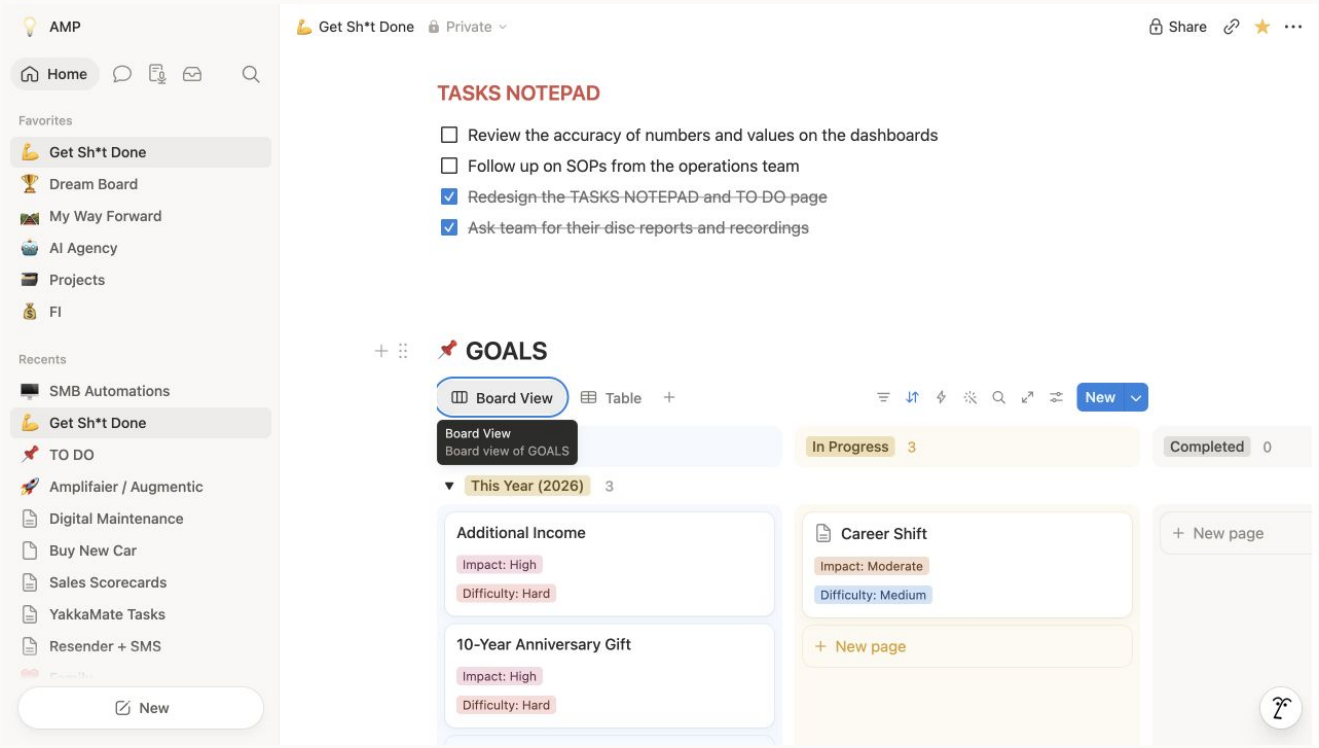


Figure 9. Board view reference: grouped columns, group totals, cards with visible properties and chips, per-column New page action and horizontal overflow.

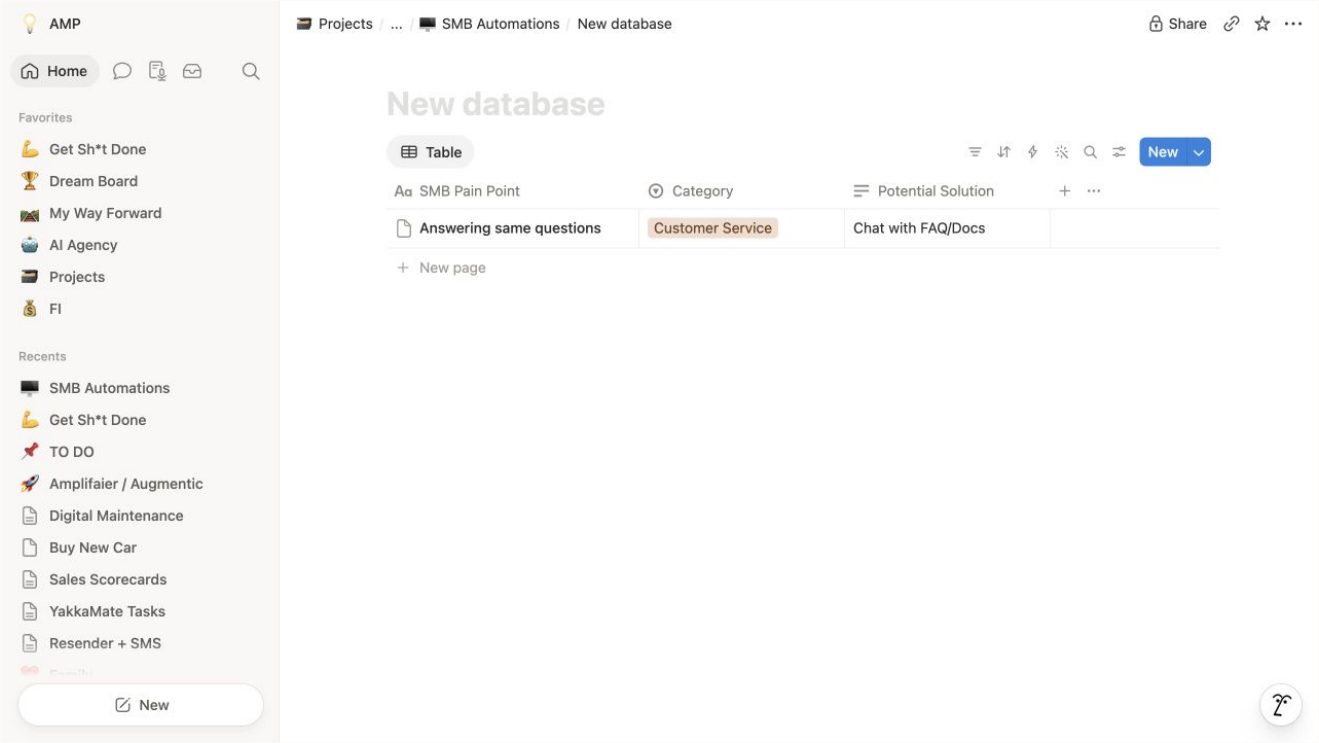


Figure 10. Verified full-page data source. Inline and full-page presentations share schema, records, view tabs and query controls rather than duplicating data.

6. Search, commands, activity and collaboration

P0 Global search

- Index page titles, block text, database properties, filenames and breadcrumbs. Apply permission filtering before result ranking and snippets.
- Modal search opens over current context, supports title-only, creator, location, date and type filters, and previews a selected result without navigation.
- Ranking blends title match, exact phrase, recency, graph proximity and user interaction. Show why a result matched.

P0 Command palette

- Search page actions, navigation, formatting and database commands. Expose keyboard shortcuts and disable commands with a clear reason.
- Dangerous actions require confirmation and identify the target. Export/import and settings remain searchable.

P1 Comments and mentions

- Page and block comment threads with resolved state, reactions, edit/delete controls, mentions and notifications.
- Unread/activity inbox groups mentions, comments, assignments, invites and page updates with mark-read controls.

P1 Sharing and publishing

- Page roles: full access, edit, comment and view. Inherit from parent unless a deliberate ACL boundary is created.
- Invite people/groups, copy internal link, link access, expiry and public publishing. Publishing must never be enabled implicitly by import.
- A permission evaluator returns both effective role and provenance so the UI can explain inherited access.

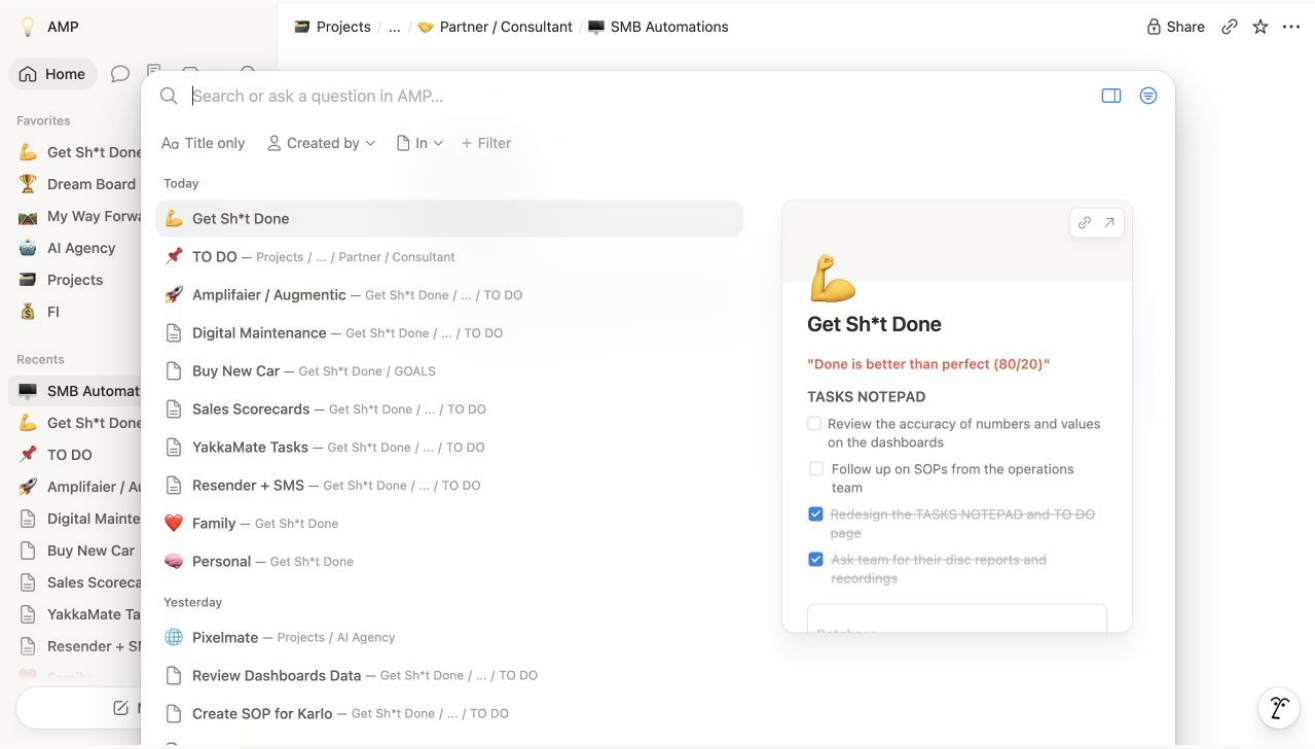


Figure 11. Search reference: persistent context underneath, filter row, date-grouped results and a right-side preview panel.

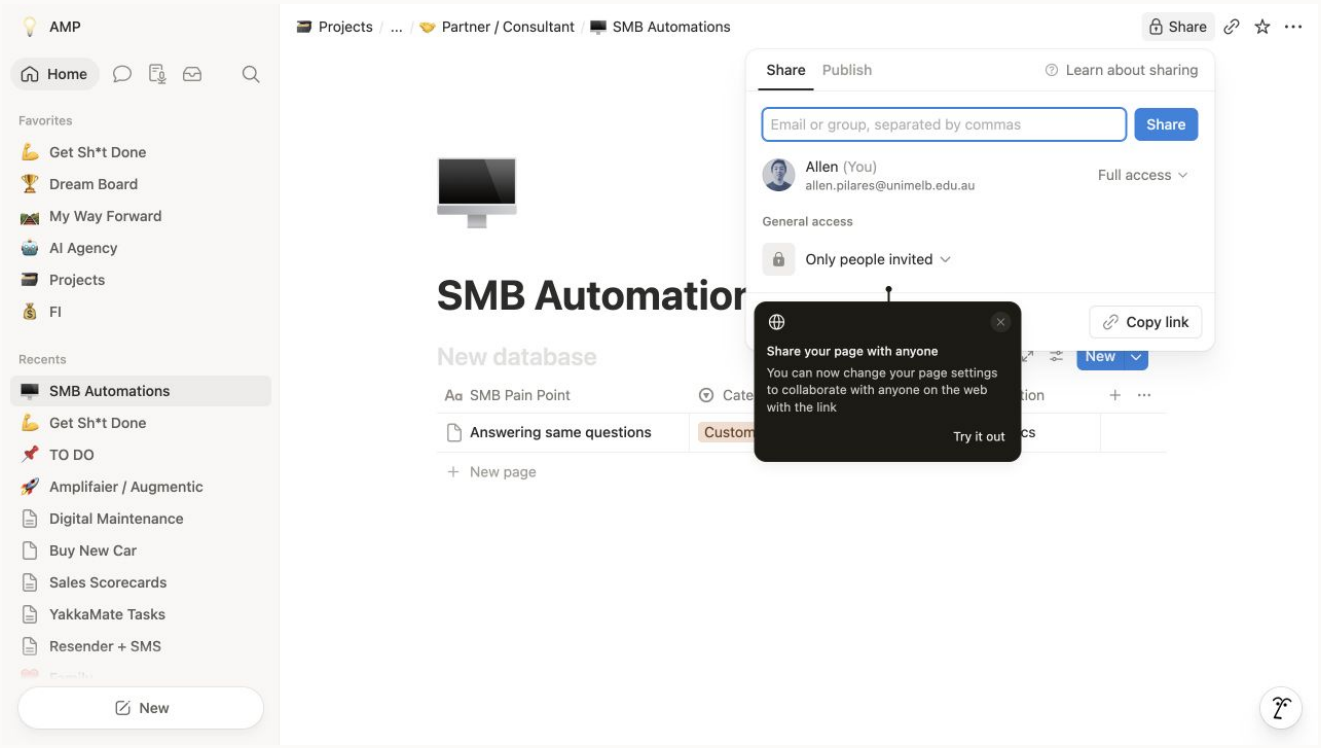


Figure 12. Share dialog reference: separate Share and Publish states, invite field, member role, general access and Copy link.

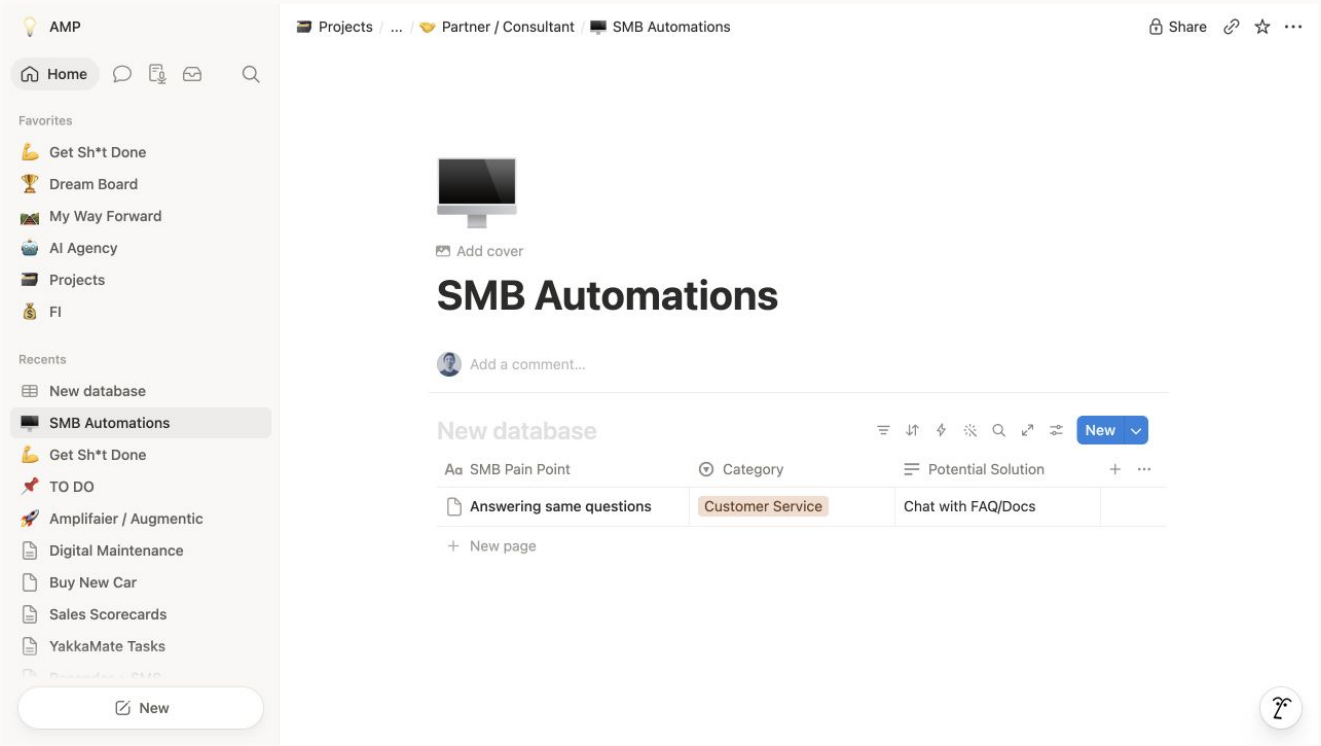


Figure 13. Verified comment composer with author context, mention and attachment entry points, and a disabled send state until valid content is present.

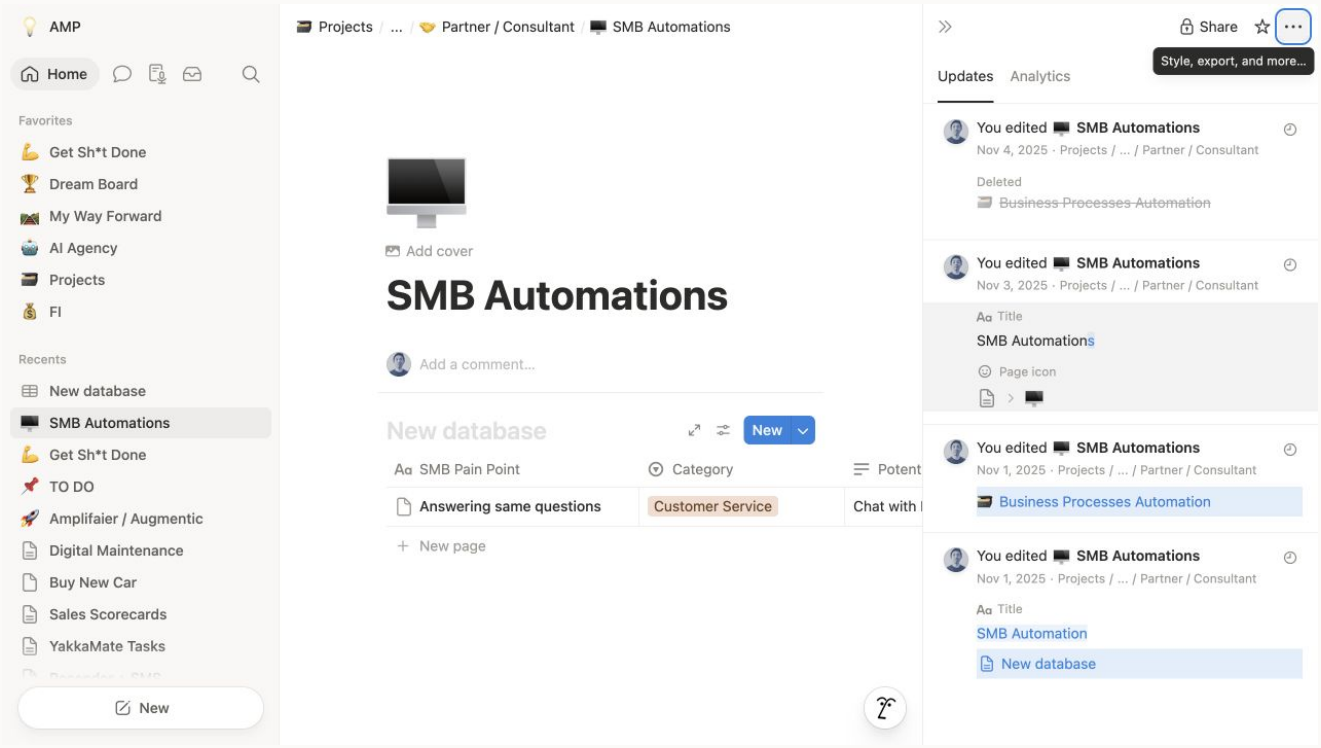


Figure 14. Verified updates panel: actor, action, linked entity, date, breadcrumb context and a jump to the relevant historical version.

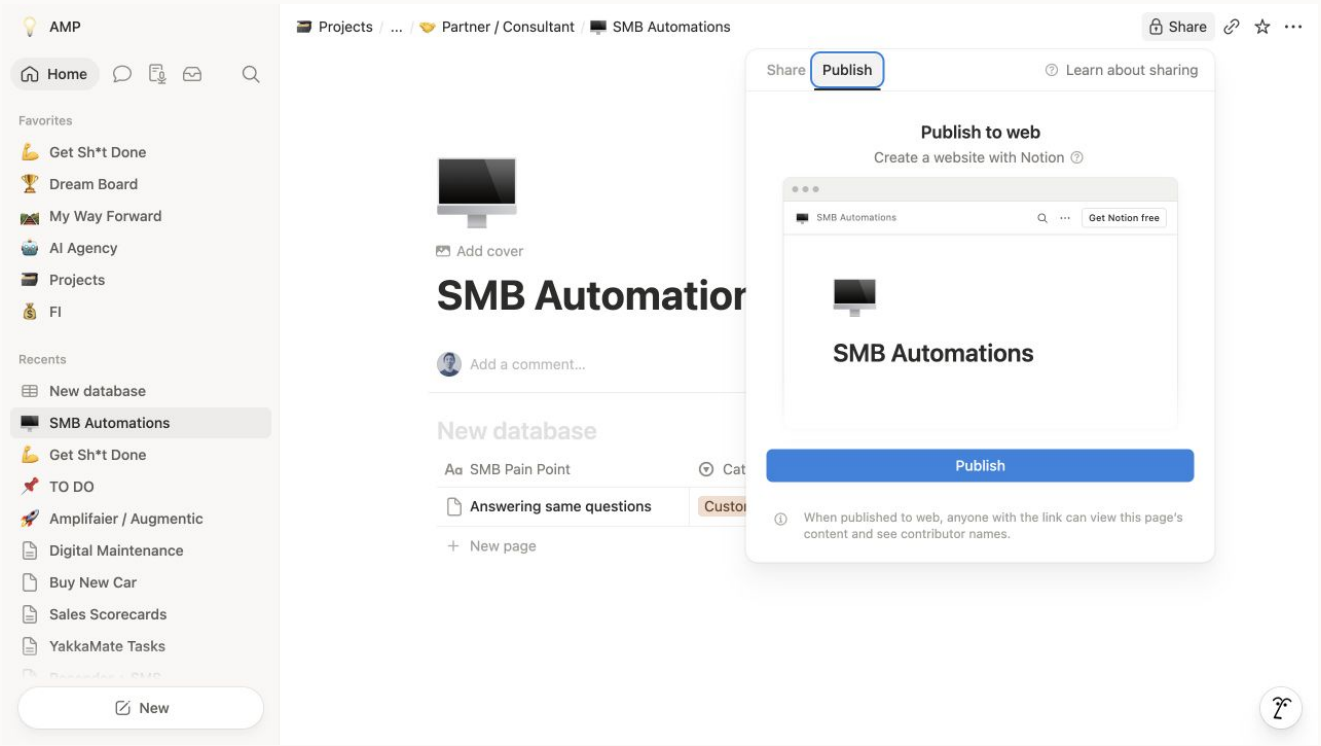


Figure 15. Verified Publish tab: public visibility and contributor-name exposure are disclosed before the explicit Publish mutation.

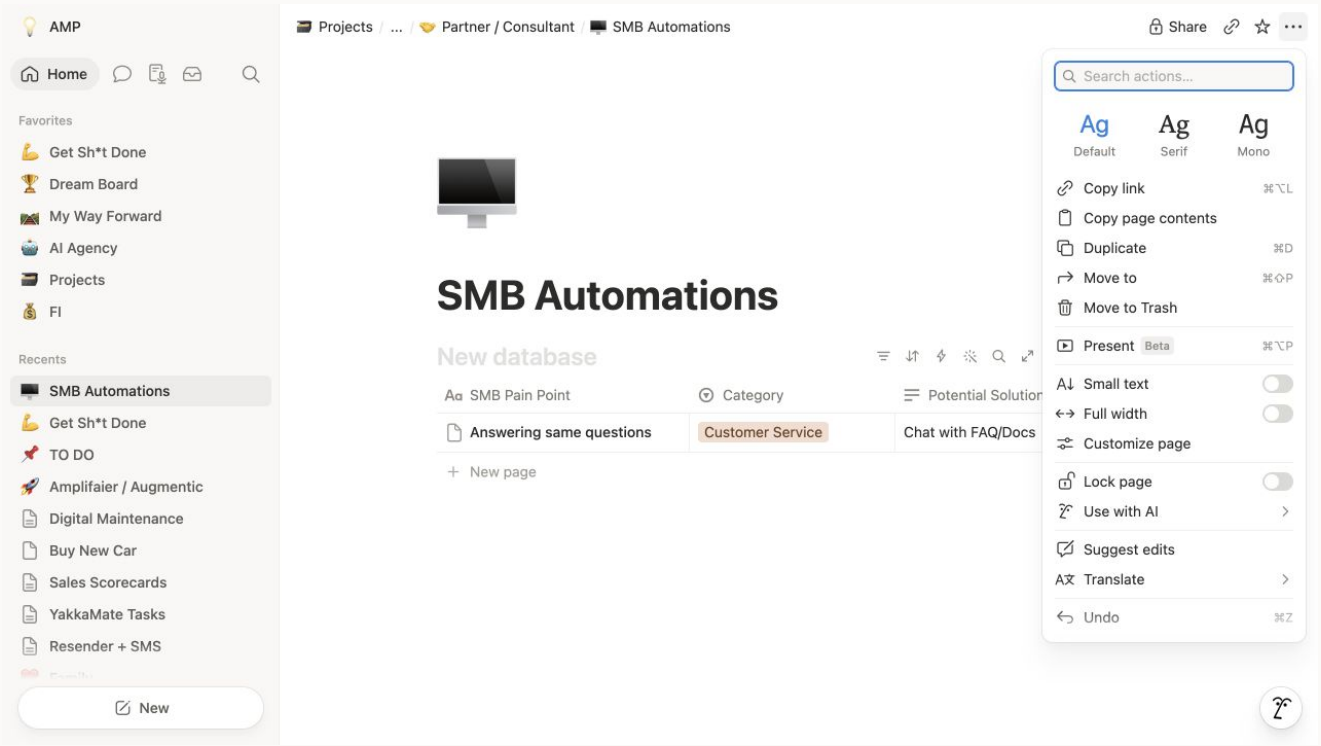


Figure 16. Page action palette reference. Group commands by intent, show shortcuts and metadata, and separate destructive actions from common formatting/navigation.

7. Migration and portability

Migration is a product, not a one-off script. Every job must be inspectable, resumable, idempotent, reversible until commit, and honest about loss.

Supported inbound paths

Source	P0 mapping	Risk handling
Notion Markdown + CSV export	Markdown files to pages/blocks; CSVs to data sources; folders to hierarchy; assets to object storage.	Infer schema conservatively; preserve raw source files and import manifest.
Notion HTML export	DOM to blocks; linked files and local hyperlinks resolved against export root.	Sanitize HTML; preserve unsupported DOM fragment as attachment plus loss item.
Notion API supplement	Pages, blocks, users, comments where authorized, page properties and data-source schema.	Respect pagination, rate limits and API omissions; never claim it is a full backup.
Generic Markdown/HTML/DOCX	Pages with supported structural blocks and assets.	Provide conversion preview and per-file warnings.
CSV/TSV/XLSX	Rows become page records; columns become properties after mapping.	User confirms inferred types, title column and duplicate policy.
ZIP	Recursive discovery of supported files; preserve relative hierarchy.	Reject traversal paths; virus scan; limits; resumable upload.

Import pipeline

Stage	Contract
1. Ingest	Hash and store immutable source archive; unpack in a sandbox; inventory files and sizes.
2. Parse	Build normalized intermediate records for workspaces, pages, blocks, schemas, rows, views, users and assets.
3. Plan	Display counts, type mappings, conflicts, unsupported features, estimated storage and choices before commit.
4. Allocate	Pre-assign Kumi IDs and build source-ID/path/URL mapping so links and relations can resolve later.
5. Write	Batch pages, blocks, schemas and properties transactionally; checkpoint each partition.
6. Resolve	Connect parent edges, internal links, relations, mentions, people, rollups and formula dependencies.
7. Assets	Deduplicate by content hash, copy bytes, retain original filename/MIME and rewrite references.
8. Verify	Compare source inventory to destination counts, hashes, link graph and property samples.
9. Report	Generate completion summary, warnings, unresolved references and downloadable machine-readable loss report.

Idempotency and rollback

- Every job has an immutable source hash and mapping table keyed by source identity. Retrying a completed unit updates the same destination entity instead of duplicating it.
- Import into a hidden staging scope. Commit makes the imported root visible atomically; rollback removes only entities owned by that job and keeps audit metadata.
- Relations and links may be unresolved during initial write. Store deferred references and retry resolution after all IDs are allocated.
- Never silently coerce mixed or unknown values. Preserve original serialized values in the import record and list every coercion in the report.

Outbound portability contract

- Workspace export contains manifest.json, pages as Markdown and/or HTML, databases as CSV plus schema.json, views.json, comments.json, permissions.json, users.json and an assets directory.
- A complete Kumi JSON export is versioned, documented and round-trips into a clean Kumi instance without changing IDs, links, relations, schema or supported view state.
- Exports are encrypted in transit, optionally encrypted at rest, expire automatically and expose progress/cancel/retry. Audit creation and download events.
- Offer a backup verifier that opens the archive independently, checks hashes and produces a human-readable inventory.

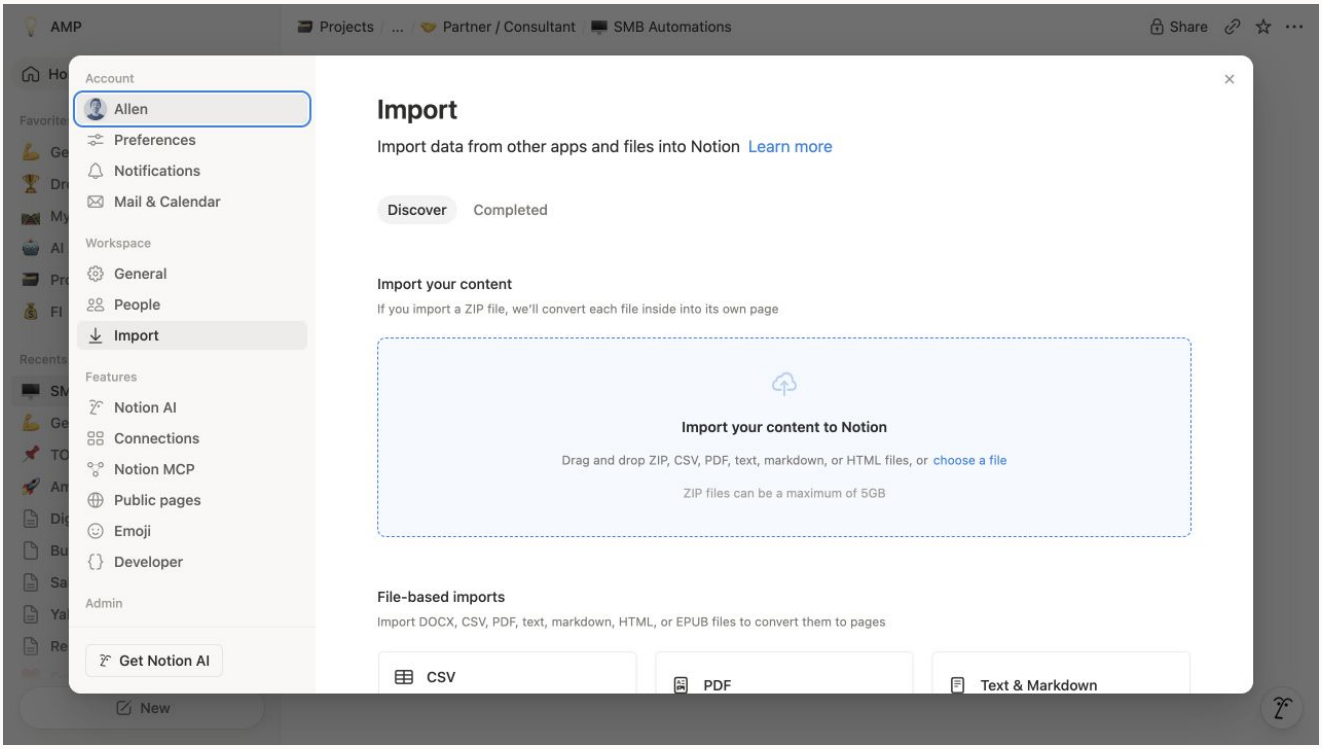


Figure 17. Observed import hub: drag/drop and file selection, ZIP/CSV/PDF/text/Markdown/HTML routes, source cards, and completed-job history.

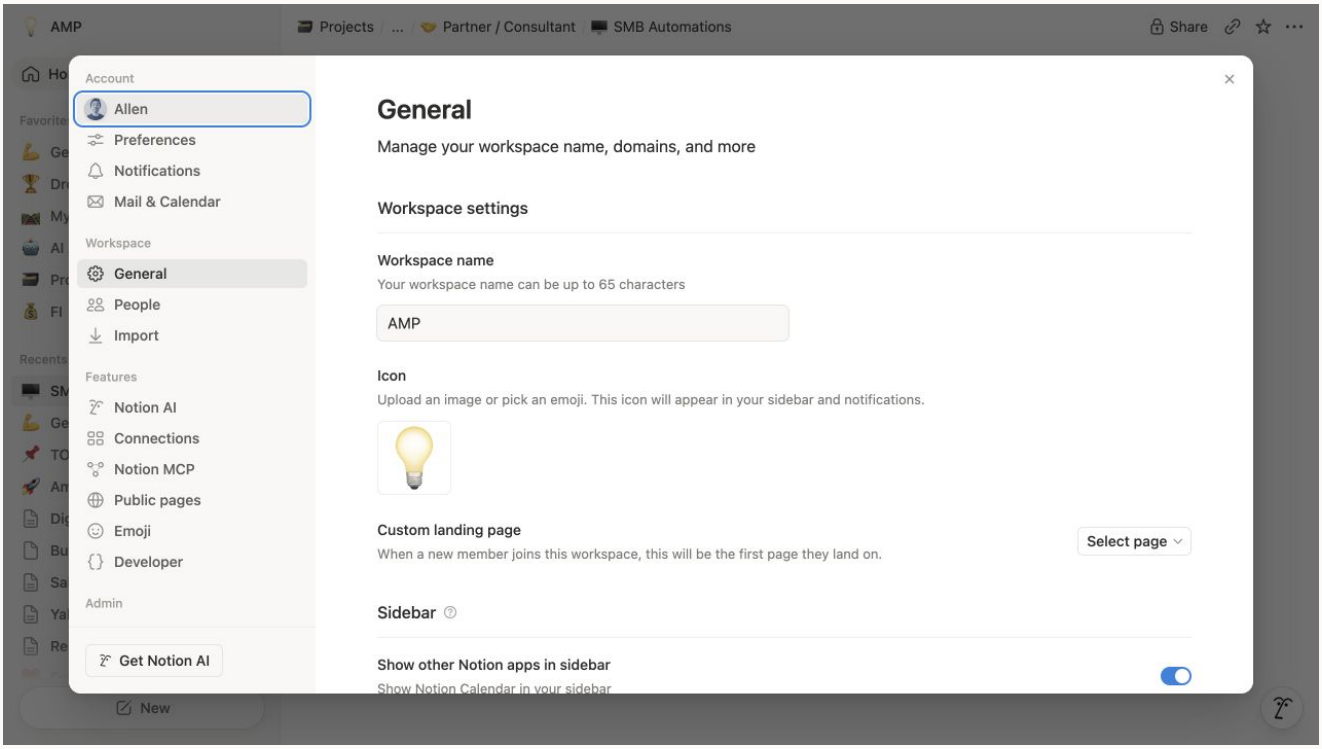


Figure 18. Workspace General reference: workspace identity, landing-page and governance controls. Export is located farther down this route, with deletion isolated in the danger zone.

8. Domain model and persistence

Core entities

Entity	Essential fields	Notes
Account/User	id, identities, profile, preferences	A user may belong to multiple workspaces.
Workspace	id, name, icon, locale, timezone, settings	Tenant and encryption boundary.
Membership/Group	user, workspace, role, group edges	Independent of page ACLs.
Page	id, workspace, parent, title, icon, cover, state	Container plus optional data-source record.
Block	id, page, parent, rank, type, payload, version	Ordered tree; payload validated by type.
DataSource	id, page, schema version, title	Schema and records separated from view.
Property	id, data source, name, type, config, rank	Stable ID survives rename.
PropertyValue	page, property, typed value	Sparse storage with typed indexes.
View	id, source, type, query, layout, owner/scope	Saved projection; optionally personal.
Asset	id, hash, MIME, size, object key, metadata	Deduplicated; signed download URLs.
ACL	resource, principal, role, inherited_from	Compute effective access with provenance.
Comment	id, target, thread, author, body, status	Page/block targets, mentions, reactions.
Revision/Operation	entity, actor, version, patch, timestamp	History, audit and conflict analysis.
ImportJob/Map	source hash, status, checkpoints, mappings	Durable migration control plane.

Storage strategy

- PostgreSQL is the source of truth for tenant, graph, schema, ACL, revisions and job state. Use row-level tenant keys on all records.
- Object storage holds assets, immutable import sources and export archives. Content hashes support deduplication and verification.
- Search uses PostgreSQL full-text at small scale or a dedicated index when ranking, snippets and permission-aware filtering require it. The database remains authoritative.
- A queue executes import/export, indexing, notification, thumbnail and automation jobs with leases, retries and dead-letter visibility.
- Cache computed permissions, page outlines and hot query results only with versioned invalidation keys.

9. Service architecture and API

Recommended shape

Component	Responsibility	Boundary
Web client	Shell, editor, views, search, settings, optimistic state	No direct database or object-store credentials.
Application API	Auth, workspaces, pages, blocks, data sources, ACL, comments	Transactional commands and permission checks.
Realtime gateway	Presence, operation broadcast, cursors, sync status	Authenticated workspace/page channels.
Migration worker	Import planning, parsing, write/resolve/verify	Sandboxed parsers; job-scoped credentials.
Export worker	Snapshot, serialize, package, hash and expire	Read-consistent snapshot with audit trail.
Search indexer	Permission-safe documents, snippets and ranking fields	Versioned events; deletion tombstones.
Asset service	Upload, scan, transform, signed delivery	Hash validation, tenant ownership and quotas.
Notification worker	Mentions, comments, invites and digests	Idempotent deliveries and user preferences.

Representative API surface

Method	Route	Purpose
GET	/v1/workspaces/:id/tree	Permission-filtered sidebar tree and revision token.
GET/PATCH	/v1/pages/:id	Read/update page metadata with optimistic version.
GET/POST/PATCH	/v1/pages/:id/blocks	Paginated block tree and batched editor operations.
POST	/v1/operations/batch	Atomic editor changes with client operation IDs.
GET/POST	/v1/data-sources/:id/query	Schema-aware query, cursor pagination and totals.
GET/PATCH	/v1/views/:id	Saved view state and lock/version checks.
POST	/v1/search	Permission-aware ranked results and snippets.
GET/POST	/v1/pages/:id/comments	Threads, replies, resolve and mentions.
GET/PATCH	/v1/resources/:id/permissions	Effective and explicit ACL management.
POST/GET	/v1/import-jobs	Upload/plan/commit/status/report/cancel/retry.
POST/GET	/v1/export-jobs	Create/status/download/expire a portable export.

Concurrency contract

- Each client operation has a unique ID, actor, base version and deterministic patch. The server acknowledges or rejects with conflict context.
- Use a CRDT or well-tested operation transform for rich text and ordered blocks. Metadata and schema mutations may use optimistic locking with explicit conflict UI.
- Presence is ephemeral; saved content is durable. Realtime transport failure must degrade to retryable HTTP sync without content loss.

10. UI system and interaction specification

Observed visual baseline

Token	Baseline	Implementation guidance
Canvas	Warm white / white	Use #FBFAF8 shell and #FFFFFF content surfaces; dark mode via semantic tokens.
Text	Charcoal	Primary near #2C2C2B; secondary #6B6A66; never rely on low contrast for essential state.
Sidebar	Approx. 270 px	Resizable 220-360 px; drawer below 900 px; state persists.
Top bar	Approx. 44 px	Breadcrumb left; collaboration/actions right; sticky without obscuring content.
Page title	Approx. 40/48 px	Responsive clamp; content max width 720 px by default; full-width option.
Radii	8-12 px	Menus/dialogs 10-12 px; controls 6-8 px; chips fully rounded or 3-5 px.
Dividers	Low contrast	1 px semantic border; stronger only for focus and selection.
Primary	Blue	Use for New/confirm/focus. Destructive actions remain red and isolated.
Motion	Subtle	120-180 ms state transitions; honor reduced-motion preference.

Behavioral details

- Menus anchor to their trigger, autofocus searchable lists where appropriate, trap focus in modal dialogs and close with Escape.
- Hover-only actions must also be keyboard reachable and discoverable on touch. Use visible focus rings that exceed the surrounding contrast.
- Inline data controls stay compact; expanded configuration uses a right-side inspector so the underlying page remains spatially stable.
- Loading preserves layout with skeletons; empty states contain one clear action; errors keep user input and offer retry/copy diagnostics.
- Responsive: desktop two-pane, tablet overlay rail plus flexible canvas, mobile single-pane navigation with full-screen editors and bottom-safe controls.

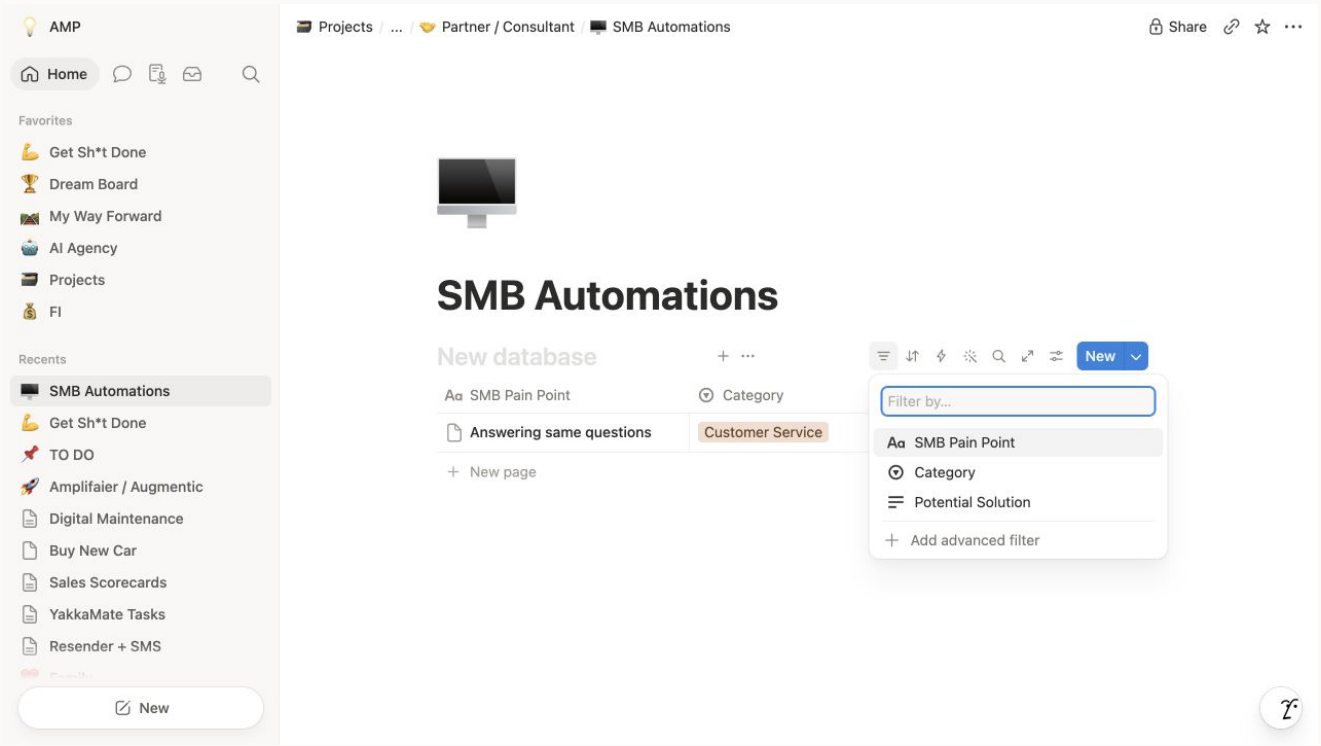


Figure 19. Compact filter interaction: anchored menu, typed property list, advanced filter path and no context switch.

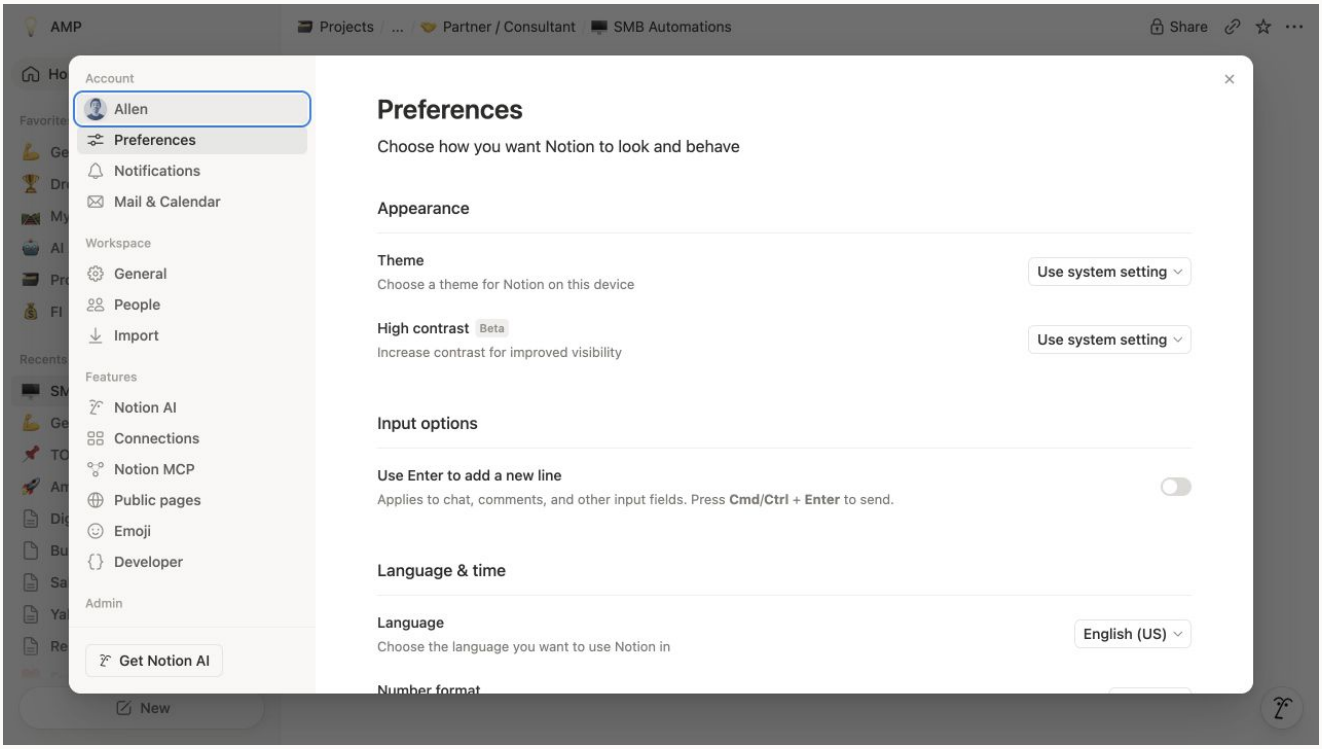


Figure 20. Preferences reference: settings use a modal shell with persistent section navigation and simple, reversible controls.

11. Security, privacy, reliability and operations

Area	Requirement
Authentication	OIDC/passwordless options, MFA, session management, device revocation and optional SSO/SCIM for enterprise.
Authorization	Deny by default; evaluate effective ACL server-side on every read/write/search/index/download path; test inheritance boundaries.
Tenant isolation	Workspace key on all data, scoped object keys, background-job tenancy, rate limits and automated cross-tenant tests.
Encryption	TLS in transit; managed encryption at rest; customer-managed keys optional later; secrets never stored in page content logs.
Assets	Validate MIME and size, malware scan, strip unsafe active content, signed expiring URLs and disposition controls.
Imports	Sandbox extraction/parsing, block path traversal, decompression limits, file-count limits and source retention controls.
Audit	Immutable events for membership, ACL, public links, exports, downloads, destructive actions and admin settings.
Backups	Encrypted automated backups, documented RPO/RTO, routine restore drills and user-verifiable exports.
Availability	Graceful degradation for search/realtime/worker outages; editor writes queue locally and expose sync state.
Observability	Structured logs without content by default, traces, metrics, job dashboards, import loss metrics and per-tenant support bundles.
Deletion	Trash retention, workspace deletion delay/confirmation, legal hold hooks and verified asset/index tombstoning.

12. Non-functional requirements

Quality	Target
Editor latency	Local keystroke feedback p95 < 50 ms; server acknowledgement p95 < 500 ms in-region.
Page load	Warm p75 < 2 s to usable; stream long block trees and retain scroll position.
Database query	p95 < 800 ms for indexed 100k-row sources under supported filter/sort combinations.
Search	p95 < 700 ms for first page; index visible edits within 5 s under normal operation.
Import	Resume after worker/process failure; no duplicate destination entities; progress at file/entity/stage level.
Export	Read-consistent snapshot; checksum manifest; predictable expiry; independent archive verifier.
Accessibility	WCAG 2.2 AA; full keyboard operation, screen-reader labels, focus management, zoom/reflow and reduced motion.
Browsers	Latest two major versions of Chrome, Safari, Firefox and Edge; responsive PWA baseline.
Localization	UTF-8, locale-aware dates/numbers, explicit timezone semantics and translatable UI strings.
Data scale	Initial design test: 100k pages/workspace, 1m blocks, 100k rows/source, 5 GB import archive.

13. Delivery plan

Phase	Scope	Exit gate
0. Foundations	Auth, workspaces, tenant model, page/block schema, design tokens, object storage, job framework.	Threat model, data model review, seeded demo workspace and CI quality gates.
1. Portable core	Sidebar/page shell, rich editor P0 blocks, page history basics, Markdown/HTML/CSV import and full Kumi export.	Round-trip test and zero silent-loss policy pass.
2. Databases	Typed properties, table/board/list/gallery/calendar/timeline, filters/sorts/groups and row page modes.	Schema/property/view acceptance suite and 100k-row performance gate.
3. Find and collaborate	Global search, comments, mentions, inbox, sharing, groups and publish controls.	Permission matrix, search leakage tests and accessibility pass.
4. Migration hardening	Notion export specialist parser, API supplement, resumable large jobs, verification and loss reports.	Representative real-workspace migrations approved by owners.
5. Expansion	Charts/forms/dashboard/map/feed, automations, integrations, offline and optional AI.	Core reliability and portability SLOs stable for two releases.

Team shape

A credible first release needs a product/design owner, 2-3 frontend/editor engineers, 2 backend/data engineers, one migration/reliability engineer, and shared QA/security support. The block editor, permissions and migration pipeline are each specialist domains; schedule explicit technical spikes before committing to a date.

14. Acceptance test matrix

ID	Scenario	Pass condition
MIG-01	Import nested Notion Markdown/CSV export with assets	Hierarchy, blocks, rows, schemas, internal links and assets match inventory; warnings are explicit.
MIG-02	Interrupt and resume a 5 GB ZIP job	No duplicates, progress resumes from checkpoint and final checksums pass.
MIG-03	Unknown block/property	Original value is retained in source record and appears in the loss report; job does not silently drop it.
EXP-01	Export then import into a clean instance	Stable IDs, supported content, relations, schemas, views and files round-trip.
ED-01	Concurrent rich-text edit by two users	Both edits converge, cursor/focus stays stable and history attributes actors.
ED-02	Reorder nested blocks offline then reconnect	No lost blocks; conflict is merged or explicitly surfaced.
DB-01	Two views share a data source	Row edit appears in both; filter/sort/layout state remains view-specific.
DB-02	Relation + rollup + formula chain	Resolution and recomputation are correct after create, update, delete and import.
ACL-01	Inherited access and private child boundary	Effective roles are correct across UI, API, search, asset and export paths.
ACL-02	Publish then unpublish	Public URL changes state immediately, caches invalidate and audit events are complete.
SEA-01	Search restricted page text	Unauthorized users receive no title, snippet, count, timing clue or cached asset.
UX-01	Keyboard-only common workflow	Create/open/edit/filter/search/comment/share actions are reachable with visible focus.
A11Y-01	Screen reader page/database flow	Landmarks, names, roles, values and dynamic announcements are meaningful.
REC-01	Restore page version	Restored state becomes a new version; later history remains accessible.

15. Product decisions still required

1. Hosting model: Kumi-managed cloud, self-hosted, or both? This changes authentication, updates, storage and support boundaries.
2. Collaboration scope at first release: single-user migration first, or multi-user sharing and realtime from day one?
3. Migration source: export archive only, optional Notion API connection, or both? Define expectations for content Notion does not export.
4. Editor engine: adopt a mature open-source block/rich-text foundation or build custom primitives? License and long-term schema control matter.
5. Formula compatibility: strict Notion-like syntax, a translated subset, or Kumi-native formulas with an import translation report?
6. Automation compatibility: preserve as inert documented definitions, translate supported triggers/actions, or exclude until a later phase?
7. Search and AI: local-only/private semantic search, hosted model integration, or no AI in the portability-first release?
8. Retention/compliance: required regions, backup retention, audit duration, legal hold, SSO/SCIM and customer-managed encryption keys.

Recommended default

Ship a single-user or small-team, cloud-deployable core first, with a documented self-hosting path, export-archive migration, table/board databases, full outbound portability, and no AI dependency. Add realtime collaboration and API-assisted migration once the loss-report and round-trip gates are consistently passing.

Appendix A. Evidence map and source notes

Browser evidence

The screenshots in this document were captured from the user's logged-in Notion workspace in a read-only pass at a 1280 x 720 CSS-pixel viewport. The exploration opened menus, dialogs and settings surfaces but did not create, edit, share, upload, download, export, invite, move, duplicate, trash, delete or toggle settings.

Official reference points

Reference	URL	Use
Notion Help - Export your content	https://www.notion.com/help/export-your-content	Current workspace/page export formats and constraints.
Notion Help - Import data into Notion	https://www.notion.com/help/import-data-into-notion	Current import sources, file mappings, limits and known losses.
Notion API - Introduction	https://developers.notion.com/reference/intro	Supported endpoint families and pagination model.
Notion API - Page properties	https://developers.notion.com/reference/page-property-values	Typed page property values and stable property identifiers.
Notion API - Block object	https://developers.notion.com/reference/block	Block-oriented representation and supported object families.
Notion API - Query a data source	https://developers.notion.com/reference/query-a-data-source	Typed filtering, sorting and data-source query semantics.

Important export limitation

Notion's current help documentation states that a complete workspace export cannot be instantly re-created by re-uploading it, and that pages outside the exporter's access may be omitted. Kumi must therefore treat an export as a lossy source that needs verification, not as proof of a complete backup. The target product should outperform this by providing a documented, round-trippable native archive.

Appendix B. Screenshot inventory

Capture	Surface	Evidence
01	Workspace shell + inline table	Navigation, page chrome, data source toolbar
02	Share permissions	Roles, general access, publish separation
03	Page actions	Command organization, import/export discoverability
04	Import hub	File/app sources and job-history model
05	Workspace General	Export, analytics and isolated danger zone
06	Preferences	Theme, locale/time and input behavior
07-08	Global search	Filters, result metadata, snippets and preview
09-15	Database controls	Views, filters, sorts, settings and property visibility
16	Rich page editor	Blocks, title spacing and inline data
17-18	Table and board	Shared data rendered by multiple saved views
19-21	Workspace/editor states	Switcher and clean editing context

End of specification

The recommended next artifact is a thin vertical-slice prototype: import one nested Markdown/CSV sample, render the workspace tree, edit blocks, switch table/board views, search, and export a round-trippable archive.